

Plan de desarrollo del videojuego “Bullet Storm Retro” en C++ (SFML + wxWidgets)

El desarrollo se basará en Programación Orientada a Objetos, definiendo clases abstractas para componentes generales (por ejemplo, una clase `Entity` base para todos los objetos del juego). Se diseña una jerarquía clara de clases (nave del jugador, enemigos, proyectiles, power-ups, escenas, etc.) usando herencia cuando convenga. Por ejemplo, la clase abstracta `Entity` puede incluir atributos comunes (posición 2D, tamaño, sprite y visibilidad) y métodos genéricos (`draw/update`), dejando `draw()` o `pintar()` como método virtual puro que implementan las subclases ¹. En esta jerarquía, las clases derivadas (p.ej. `MovableEntity`, `Player`, `Enemy`, `Bullet`, `PowerUp`) añaden datos y comportamientos específicos.

```
// Entity.h: clase base abstracta para cualquier objeto del juego
class Entity {
protected:
    sf::Sprite sprite;
    sf::Vector2f position;
public:
    Entity(const sf::Texture& tex, float x, float y) {
        sprite.setTexture(tex);
        sprite.setPosition(x, y);
    }
    virtual void update(float dt) = 0; // Lógica del
objeto
    virtual void draw(sf::RenderWindow& window) { // Dibuja el
sprite
        window.draw(sprite);
    }
    virtual sf::FloatRect getBounds() const { // Box para
colisiones
        return sprite.getGlobalBounds();
    }
    virtual ~Entity() {}
};
```

Dentro de esta jerarquía, la clase `Player` derivará de la clase móvil base (por ejemplo `MovableEntity` o directamente `Entity`) e incluirá atributos como número de vidas, velocidad, e implementará la capacidad de disparar. Según referencias académicas, la clase Jugador añade “la capacidad de disparar balas” sobre la funcionalidad de la entidad móvil ². De forma similar, se define una clase `Enemy` (o varias para distintos patrones), y una clase `Bullet` (proyectil) con su propia lógica de movimiento. La organización en archivos fuente contempla separar cada clase en su `.h` y `.cpp`, por ejemplo `Player.h/.cpp`, `Enemy.h/.cpp`, `Bullet.h/.cpp`, `PowerUp.h/.cpp`, etc. Esto facilita la mantenibilidad y el trabajo en equipo.

Escenas y gestión de estados (Game States)

Se utiliza el patrón de estados para controlar diferentes pantallas del juego. Se define una clase abstracta `Scene` con métodos virtuales como `handleEvent()`, `update()` y `draw()`, que sirven de interfaz común para cada escena del juego (menú principal, escena de juego, game over, etc.). Cada escena concreta (por ejemplo `MainMenuScene`, `GameScene`, `GameOverScene`) hereda de `Scene` e implementa estos métodos según corresponda. El bucle principal (`main()`) mantiene un puntero a la escena actual y la actualiza/dibuja en cada iteración, cambiando la escena según eventos (ej. al comenzar el juego o cuando termina).

```
// Scene.h: clase abstracta base para estados del juego
class Scene {
public:
    virtual void handleEvent(const sf::Event& event) = 0;
    virtual void update(float dt) = 0;
    virtual void draw(sf::RenderWindow& window) = 0;
    virtual ~Scene() {}
};
```

Ejemplo de uso en `main.cpp` (bucle principal con cambio de escena):

```
int main() {
    sf::RenderWindow window(sf::VideoMode(800, 600), "Bullet Storm Retro");
    std::unique_ptr<Scene> currentScene = std::make_unique<MainMenuScene>();
    sf::Clock clock;
    while (window.isOpen()) {
        float dt = clock.restart().asSeconds();
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close();
            currentScene->handleEvent(event);
        }
        currentScene->update(dt);
        window.clear();
        currentScene->draw(window);
        window.display();
        // Ejemplo de transición de escena:
        // if (currentScene->isFinished()) currentScene =
std::make_unique<GameScene>();
    }
    return 0;
}
```

Separación de lógica vs. renderizado

Se mantiene claramente separada la lógica del juego del renderizado. Los métodos `update(dt)` de cada objeto (o escena) se encargan de la lógica: mover entidades, manejar entradas, detectar

colisiones, actualizar puntajes, etc. Por el contrario, los métodos `draw(window)` solo invocan llamadas de SFML para renderizar sprites y texto en pantalla. De este modo, no se mezcla lógica de juego con código de la GUI o el motor gráfico. Por ejemplo, `Entity::update()` movería la posición del sprite pero no lo dibuja, y sólo `draw()` llama a `window.draw(sprite)`. Este enfoque coincide con prácticas académicas donde el método `pintar()` es abstracto y separado de la lógica interna ¹. Además, la ventana SFML se gestiona sólo dentro del bucle principal del juego, y no se realiza lógica adicional en componentes de interfaz (como wxWidgets).

Jugador y disparo continuo

La clase `Player` deriva de la clase base de entidades y gestiona las vidas (inicialmente 3), la velocidad, y la mecánica de disparo. Por ejemplo, se puede llevar un contador interno de cooldown para disparo continuo (rapidez de fuego). En cada ciclo de `update(dt)`, el jugador lee el teclado (p.ej. flechas para moverse y barra espaciadora para disparar). Cuando el jugador dispara, se crea un nuevo `Bullet` que se agrega a la lista de proyectiles del jugador. Según la referencia, el jugador almacena sus balas en un vector con atributos de velocidad, posición, tamaño y estado vivo ².

```
// Player.h: jugador con 3 vidas y disparo continuo
class Player : public Entity {
private:
    int lives = 3;
    float speed = 200.0f;
    float shootCooldown = 0.2f;
    float shootTimer = 0.0f;
public:
    void handleInput() {
        // Movimiento horizontal con flechas
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left))
            sprite.move(-speed * dt, 0);
        else if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right))
            sprite.move(speed * dt, 0);
        // Disparo continuo con espacio
        if (shootTimer <= 0 &&
            sf::Keyboard::isKeyPressed(sf::Keyboard::Space)) {
            shoot();
            shootTimer = shootCooldown;
        }
    }
    void update(float dt) override {
        handleInput();
        // Actualizar temporizador de disparo
        if (shootTimer > 0) shootTimer -= dt;
        // Otras lógicas (colisiones, etc.)...
    }
    void shoot() {
        // Crear bala y agregarla a la escena de juego
    }
    void loseLife() { lives--; }
```

```
int getLives() const { return lives; }
};
```

Según la documentación interna del curso, “la clase Jugador añade la capacidad de disparar balas” y gestiona sus propios proyectiles ². Al perder todas las vidas, la escena de juego cambia a “Game Over”.

Enemigos y patrones de disparo (bullet-hell)

Se definen múltiples enemigos con IA simple, cada uno disparando proyectiles en patrones tipo bullet-hell. Por ejemplo, se puede crear una clase base `Enemy` (derivada de `Entity`) y luego varias clases derivadas (enemigo tipo A, jefe, etc.) que implementan distintos esquemas de disparo. Los patrones pueden ser circular, espiral, en abanico, etc., calculando ángulos en base a trigonometría. En cada `update(dt)`, un enemigo puede llamar a su método `shoot()`, generando uno o varios `Bullet` con velocidades dirigidas.

```
// Enemy.h: clase base para enemigos
class Enemy : public Entity {
public:
    void update(float dt) override {
        // Ejemplo: moverse verticalmente
        sprite.move(0, 50 * dt);
        // Disparar periódicamente
        shootTimer -= dt;
        if (shootTimer <= 0) { shootTimer = shootCooldown; shoot(); }
    }
    virtual void shoot() {
        // Patrón básico de disparo (heredado o sobrescrito)
    }
};
```

```
// Bullet.h: proyectil con velocidad
class Bullet : public Entity {
private:
    sf::Vector2f velocity;
public:
    Bullet(const sf::Texture& tex, const sf::Vector2f& pos, const
sf::Vector2f& vel)
        : Entity(tex, pos.x, pos.y), velocity(vel) {}
    void update(float dt) override {
        sprite.move(velocity * dt); // pos += vel * dt (ver ejemplo 3)
    }
};
```

De manera análoga al jugador, los enemigos pueden almacenar sus proyectiles en un vector. El diseño orientado a objetos facilita crear distintos comportamientos en cada subclase `Enemy` sin duplicar código. La referencia sobre movimiento de proyectiles muestra justamente cómo actualizar su posición con `velocidad * dt` ³.

Sistema de colisiones

Para detectar colisiones se puede usar la caja delimitadora (bounding box) de cada sprite, accesible con `getGlobalBounds()` de SFML. Por ejemplo, en cada ciclo se comprueba intersecciones con: `if (player.getBounds().intersects(enemy.getBounds())) { ... }`. De forma similar se detecta si un `Bullet` del jugador choca con un enemigo, o un `Bullet` enemigo con el jugador. Al detectarse colisión, se aplican efectos: quitar vidas, destruir objetos o recoger power-ups. Se recomienda manejar estas lógicas en `update(dt)`, no en eventos de GUI.

```
// Ejemplo de colisión dentro de GameScene::update
for (auto& bullet : playerBullets) {
    for (auto& enemy : enemies) {
        if (bullet->getBounds().intersects(enemy->getBounds())) {
            enemy->destroy();
            bullet->destroy();
            score += 100;          // Sumar puntos
            // Posible spawn de power-up...
        }
    }
}
```

Puntuación y Power-ups

Se mantiene un contador de puntos (`score`) incrementado al destruir enemigos (por ejemplo `score += 100`). En pantalla se puede mostrar con un `sf::Text` actualizado cada frame. Además, al destruir enemigos puede aparecer un *power-up* (objeto especial). Por ejemplo, usando una probabilidad aleatoria tras cada muerte. La clase `PowerUp` (derivada de `Entity`) puede tener distintos tipos: vida extra, disparo rápido, escudo temporal, etc. Al colisionar el jugador con un power-up, se aplica su efecto (`player.addLife()`, `player.activateRapidFire()`, etc.) y se elimina el power-up. Un ejemplo educativo menciona la clase “Tesoro” como power-up que al recogerse “aumenta la puntuación del jugador” ⁴. Podemos implementar algo similar (p.ej. coin aumenta score, corazón aumenta vida) siguiendo esa idea.

```
// PowerUp.h: ejemplo de power-up
enum class PowerUpType { ExtraLife, RapidFire, Shield };
class PowerUp : public Entity {
private:
    PowerUpType type;
public:
    PowerUp(PowerUpType t, const sf::Vector2f& pos, const sf::Texture& tex)
        : Entity(tex, pos.x, pos.y), type(t) {}
    void update(float dt) override { /* Se mueve o flota */ }
    void applyTo(Player& player) {
        switch(type) {
            case PowerUpType::ExtraLife: player.addLife(); break;
            case PowerUpType::RapidFire: player.enableRapidFire(); break;
            case PowerUpType::Shield: player.enableShield(); break;
        }
    }
}
```

```

    }
};

```

Aquí, por ejemplo, al destruir un enemigo podríamos hacer:

```

if (enemyDestroyed && randFloat() < 0.2f) {
    // 20% de probabilidad de soltar vida extra
    powerUps.push_back(std::make_unique<PowerUp>(PowerUpType::ExtraLife,
    enemyPos, textureVida));
}

```

y luego detectar la recolección con la colisión del jugador y aplicar `applyTo()`.

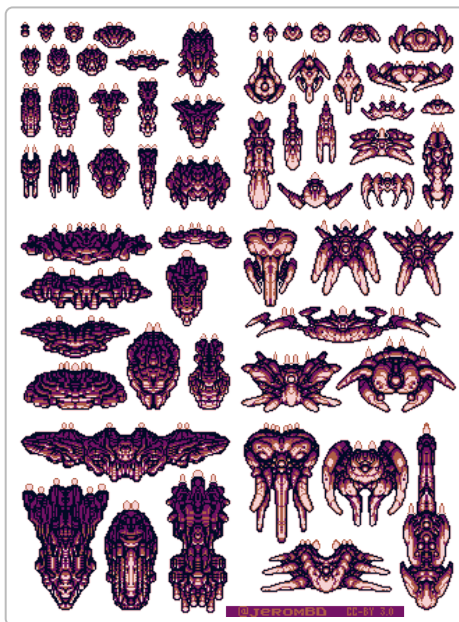


Figura: Ejemplo de sprites retro de naves espaciales (pixel art). El estilo gráfico planeado es retro, con sprites pixelados tanto para la nave del jugador como para los enemigos y proyectiles. La figura muestra un set de sprites clásicos; el juego utilizará recursos similares (se pueden obtener de repositorios libres) que encajen con esta estética.

Interfaz de configuración (wxWidgets)

Para el lanzador o menú previo se emplea wxWidgets. Se puede crear una ventana independiente (diálogo o frame) donde el jugador ingrese su nombre, seleccione dificultad, etc. Al hacer clic en "Iniciar" se cierran esos controles de wxWidgets y se inicia la ventana SFML con el juego. Por ejemplo:

```

// Launcher.h: ventana wxWidgets de configuración previa
class LauncherFrame : public wxFrame {
public:
    LauncherFrame() : wxFrame(nullptr, wxID_ANY, "Configuración") {
        // Agregar controles (wxTextCtrl para nombre, wxChoice para
        // dificultad, wxButton para iniciar, etc.)
    }
};

```

```
// Al pulsar "Iniciar", leer valores y cerrar esta ventana, luego
llamar a runGame(playerName, difficulty);
}
};
```

Esta GUI no debe mezclarse con el render de SFML: simplemente recoge opciones y luego crea la instancia del juego SFML (por ejemplo, invocando la función `main()` del juego). No se integra el canvas SFML dentro de wxWidgets, sino que son procesos separados.

Estructura de carpetas del proyecto

Se recomienda la siguiente organización de carpetas para Windows (en entorno ZinjaI o similar):

- `/src/` – Código fuente C++ (.cpp, .h). Dentro pueden subcarpetas:
- `/src/scenes/` – `Scene.h/.cpp`, `MainMenuScene.h/.cpp`, `GameScene.h/.cpp`, `GameOverScene.h/.cpp`, etc.
- `/src/entities/` – `Entity.h/.cpp`, `Player.h/.cpp`, `Enemy.h/.cpp`, `Bullet.h/.cpp`, `PowerUp.h/.cpp`, etc.
- `main.cpp` – Contiene `main()` que inicia wxWidgets o SFML según corresponda.
- `/assets/` – Recursos del juego (imágenes de sprites, fuentes de texto, sonidos). Por ejemplo:
- `/assets/sprites/` – Imágenes .png de la nave, enemigos, balas, ítems.
- `/assets/fonts/` – Archivos .ttf para textos (score, mensajes).
- `/include/` (opcional) – Si se prefiere separar headers.
- `/lib/` – Bibliotecas externas compiladas (SFML, wxWidgets).
- `/build/` o `/bin/` – Directorios de compilación o ejecutables generados.

Cada archivo .h tiene su .cpp correspondiente, siguiendo buenas prácticas de C++ (como exige el curso). Esto permite compilar todo en ZinjaI con perfiles de compilación para Windows.

Depuración en ZinjaI

ZinjaI (IDE usado en UNL) proporciona un depurador integrado basado en GDB con potentes herramientas. Entre las más útiles para este proyecto destacan ⁵:

- **Breakpoints condicionales y watchpoints:** se pueden establecer puntos de quiebre normales o condicionados (ej. parar cuando `score > 1000`) y observar variables o incluso expresiones (incluyendo rangos de array).
- **Ejecución paso a paso y backtrace:** permite avanzar línea por línea o por instrucciones ensamblador, y ver la pila de llamadas para diagnosticar errores.
- **Inspección de objetos y estructuras:** ZinjaI muestra los atributos de los objetos en forma jerárquica, incluyendo vectores y matrices en tablas amigables ⁵. Por ejemplo, al depurar el vector de enemigos o las propiedades de un `Bullet`, se visualizan sus campos internos.
- **Diagrama de flujo y trazado de código:** herramientas gráficas para entender caminos de ejecución (aunque no es central en este juego, es una capacidad adicional).
- **Consola interactiva y watch:** permite evaluar expresiones durante la pausa, útil para verificar lógica (p.ej. comprobar rangos de colisiones o valor de vidas).

En resumen, el uso de ZinjaI facilita detectar bug de lógica (pérdida incorrecta de vidas, colisiones que no se detectan, variables de tiempo mal inicializadas, etc.) al poder observar paso a paso lo que ocurre en tiempo de ejecución ⁵.

Cada parte del juego (movimientos, disparos, colisiones) debe probarse con estos instrumentos. Por ejemplo, poner un breakpoint dentro de `GameScene::update` ayuda a verificar que los límites de la ventana se respetan, o que las listas de proyectiles se actualizan correctamente. Las herramientas de depuración de ZinjaI permitirán seguir la ejecución del juego y validar los estados de los objetos en cada frame.

Este plan integral combina diseño orientado a objetos, separación clara de capas (lógica del juego vs GUI), uso de escenas para estados del juego, y buenas prácticas de archivo. Con la ayuda de ejemplos de código y de las referencias citadas, el proyecto cumple los requisitos académicos (facultad, Windows) y aprovecha las herramientas de ZinjaI para depurar eficazmente ¹ ². Se deja margen para agregar arte y audio específicos en etapas posteriores, siguiendo siempre la arquitectura establecida.

Fuentes: La estructura de clases y ejemplos de entidades provienen de prácticas docentes en desarrollo de juegos (e.g. se basa en clases genéricas de entidad ¹ y jugabilidad ²), mientras que la lógica de movimiento de proyectiles toma como modelo un tutorial de SFML ³. La información sobre depuración se obtuvo de la documentación de ZinjaI ⁵. Todos los conceptos aquí expuestos se integran en el estilo didáctico recomendado por el curso.

¹ ² ⁴ (PDF) Desarrollo de videojuegos en 2D usando Simple DirectMedia
https://www.academia.edu/99262263/Desarrollo_de_videojuegos_en_2D_usando_Simple_DirectMedia

³ Make a bullet hell game, free project tutorial
<https://glusoft.com/sfml-tutorials/make-bullet-hell-game/>

⁵ sv-Incs
<http://sedici.unlp.edu.ar/bitstream/handle/10915/20953/2836.pdf?sequence=1>